1. Purpose

This API Specification translates the Technical Design, Agent Specification, and Data Model into a concrete backend API contract. It defines resources, endpoints, request/response behavior, lifecycle states, validation, errors, and the boundaries between independent evaluation, debate, and final decision stages.

2. API Principles

Use REST-style HTTP APIs with JSON responses. Use multipart/form-data for document uploads. Keep LLM calls server-side. Never expose provider API keys to the browser. Every evaluation-specific endpoint must enforce evaluation ownership and isolation. Long-running evaluation stages should be asynchronous from the frontend perspective, with status polling or server-sent progress events as an optional enhancement.

3. Base URL and Versioning

Development: /api/v1. Production should use a versioned API prefix such as /api/v1. All response objects should include stable IDs. Breaking changes require a new API version.

4. Standard Response Envelope

Successful responses should use a consistent structure where practical: { data: ..., meta: ... }. Errors should use: { error: { code: "...", message: "...", details: ... } }. The frontend must never depend on parsing human-readable LLM prose to determine state.

5. Create Evaluation

POST /api/v1/evaluations. Purpose: create a new candidate evaluation. Request: { candidate_id, job_id }. Response: evaluation id, status=CREATED, timestamps. Validation: candidate and job must exist; user/session must be authorized.

6. Get Evaluation

GET /api/v1/evaluations/{evaluation_id}. Returns evaluation metadata and pipeline status. Response includes: id, candidate_id, job_id, status, pipeline_version, started_at, completed_at, created_at, and stage_status.

7. Upload Candidate Document

POST /api/v1/evaluations/{evaluation_id}/documents. Content-Type: multipart/form-data. Fields: file and document_type. document_type: RESUME, TRANSCRIPT, OTHER. Validate file type and size. Store the file securely and return document metadata plus processing status.

8. List Documents

GET /api/v1/evaluations/{evaluation_id}/documents. Returns document IDs, type, filename, processing status, and timestamps. Do not expose storage credentials or internal filesystem paths.

9. Process Documents

POST /api/v1/evaluations/{evaluation_id}/documents/process. Extracts text and source metadata. Response may be asynchronous: status=PROCESSING. GET /api/v1/evaluations/{evaluation_id} should expose progress.

10. Candidate Profile

GET /api/v1/evaluations/{evaluation_id}/profile. Returns the neutral Candidate Profile including profile items and Evidence IDs. It must not contain agent recommendations or final hiring judgments.

11. Build / Rebuild Profile

POST /api/v1/evaluations/{evaluation_id}/profile/build. Runs the Candidate Profile Builder from approved source documents. Response returns job status or completed profile. The profile builder must preserve the distinction between FACT, CANDIDATE_CLAIM, INFERENCE, and UNKNOWN.

12. Job Requirements

GET /api/v1/evaluations/{evaluation_id}/requirements. Returns extracted or configured job requirements. POST /api/v1/evaluations/{evaluation_id}/requirements/extract creates requirements from the job description. Each requirement has id, text, category, priority, and evidence mapping status where available.

13. Evidence

GET /api/v1/evaluations/{evaluation_id}/evidence. Returns evidence records for the evaluation, optionally filtered by document_id, validation_status, evidence_type, or requirement_id. Evidence must include source location and exact quote or structured fact where available.

14. Evidence Detail

GET /api/v1/evidence/{evidence_id}. Returns one evidence item and its source metadata. The service must verify that the requesting evaluation context is authorized to access it.

15. Run Independent Evaluation

POST /api/v1/evaluations/{evaluation_id}/agents/run-independent. Starts the four independent evaluator calls. The server must execute separate LLM calls for Technical, HR/Culture, Hiring Manager, and Skeptic. The independent prompt context for each call must exclude every other agent's output.

16. Independent Agent Status

GET /api/v1/evaluations/{evaluation_id}/agents. Returns status for each evaluator: NOT_STARTED, RUNNING, COMPLETED, FAILED, or UNAVAILABLE. Include model/prompt version and timestamps where appropriate, but do not expose internal secrets.

17. Agent Evaluation Detail

GET /api/v1/evaluations/{evaluation_id}/agents/{agent_type}. Returns the selected agent's structured evaluation, findings, confidence, recommendation, and evidence references. agent_type values: technical, hr_culture, hiring_manager, skeptic.

18. Independent Isolation Audit

GET /api/v1/evaluations/{evaluation_id}/agents/audit. Optional internal/demo endpoint. Returns execution metadata proving which shared inputs were supplied to each independent agent and confirming that other agent conclusions were excluded. Do not expose confidential prompts or API credentials.

19. Start Debate

POST /api/v1/evaluations/{evaluation_id}/debate. Preconditions: independent stage must be complete or the application must explicitly support partial mode. Creates/starts the debate. Request may include max_rounds, default 3.

20. Debate Status

GET /api/v1/evaluations/{evaluation_id}/debate. Returns debate status, current round, max rounds, start/completion times, and participant statuses.

21. Run Debate Round

POST /api/v1/evaluations/{evaluation_id}/debate/rounds. Request: { round_number }. The server loads independent evaluations and prior debate messages according to the debate rules, then generates substantive challenges/responses. It must not silently skip the required direct-response relationship.

22. Debate Messages

GET /api/v1/evaluations/{evaluation_id}/debate/messages. Supports filters by round, agent_type, message_type, and responding_to_message_id. Each message includes message_id, round_number, agent_type, type, statement, responding_to_message_id, evidence_ids, confidence, and timestamp.

23. Opinion Revisions

GET /api/v1/evaluations/{evaluation_id}/revisions. Returns original and revised positions, confidence changes, reason, triggering message IDs, and evidence IDs. Original independent conclusions must remain unchanged in the database.

24. Run Final Decision

POST /api/v1/evaluations/{evaluation_id}/decision. Preconditions: independent evaluation and debate stages must be complete according to configured policy. Invokes the separate Final Decision Agent. It must not calculate the recommendation by averaging or majority voting.

25. Final Decision

GET /api/v1/evaluations/{evaluation_id}/decision. Returns recommendation, confidence, rationale, decision factors, supporting evidence, concerns, resolved disagreements, unresolved disagreements, and alternative considered.

26. Final Report

GET /api/v1/evaluations/{evaluation_id}/report. Returns the complete report data needed by the frontend: candidate profile summary, recommendation, confidence, strengths, concerns, agent evaluations, debate, revisions, unresolved disagreements, and traceability information.

27. Traceability

GET /api/v1/evaluations/{evaluation_id}/traceability/{target_type}/{target_id}. Returns the evidence graph for a target such as a decision factor, finding, debate message, or final decision. Example chain: final decision → decision factor → finding → evidence → source document.

28. Pipeline Convenience Endpoint

POST /api/v1/evaluations/{evaluation_id}/run. Optional convenience endpoint for a full evaluation. Internally it must execute the same controlled stages: document readiness → profile → requirements → independent agents → evidence validation → debate → final decision. It must never collapse these stages into one unconstrained LLM call.

29. Status / Progress Events

GET /api/v1/evaluations/{evaluation_id}/events. Returns execution events for the interactive UI. Events include stage transitions, agent started/completed, evidence validation, debate rounds, revisions, and decision completion. This can later be replaced or supplemented by Server-Sent Events.

30. Request Validation

Reject unsupported file types, oversized files, malformed UUIDs, invalid enums, missing required fields, invalid agent types, invalid debate rounds, and requests against resources outside the evaluation scope. Use typed request/response schemas.

31. Error Codes

Recommended codes: EVAL_NOT_FOUND, DOCUMENT_INVALID, DOCUMENT_TOO_LARGE, DOCUMENT_PROCESSING_FAILED, PROFILE_NOT_READY, REQUIREMENTS_NOT_READY, INDEPENDENT_STAGE_INCOMPLETE, AGENT_FAILED, AGENT_OUTPUT_INVALID, EVIDENCE_INVALID, DEBATE_NOT_READY, DEBATE_FAILED, DIRECT_RESPONSE_MISSING, DECISION_NOT_READY, DECISION_FAILED, TRACE_NOT_FOUND, UNAUTHORIZED, FORBIDDEN, RATE_LIMITED, INTERNAL_ERROR.

32. HTTP Status Codes

200 for successful reads/updates; 201 for resource creation; 202 for accepted asynchronous jobs; 400 for malformed requests; 401 for unauthenticated requests where authentication exists; 403 for unauthorized access; 404 for missing resources; 409 for invalid lifecycle transitions; 413 for oversized uploads; 422 for schema/validation failures; 429 for rate limiting; 500 for unexpected server errors.

33. Lifecycle Enforcement

The API must prevent invalid transitions. Example: POST /agents/run-independent should reject if required documents/profile are unavailable. POST /debate should reject before independent evaluation completion. POST /decision should reject if required debate completion is missing. The exact policy for partial-agent failures must be configurable and visible in the evaluation status.

34. Security Boundary

All LLM-provider calls occur in the backend. Candidate documents are treated as untrusted data. Uploaded text must never override system or application instructions. API responses should avoid returning secrets, raw provider metadata, internal prompts, or unnecessary candidate data.

35. Concurrency Rules

Independent agents may execute concurrently because their contexts are isolated. The API should prevent duplicate concurrent independent runs for the same evaluation unless an explicit rerun/version mechanism is implemented. Debate rounds should execute sequentially because later messages depend on earlier messages.

36. Idempotency

Resource creation and long-running stage endpoints should support idempotency where duplicate requests could cause duplicate LLM calls or records. A repeated request with the same idempotency key should return the existing operation/result when safe.

37. Rate Limits and Cost Controls

Apply server-side rate limits to expensive stage endpoints. Configure maximum retries, maximum debate rounds, maximum upload size, and LLM token limits. Do not automatically retry indefinitely.

38. Frontend Data Flow

New Evaluation → upload documents → process → profile → requirements → run independent → show four isolated agent results → debate timeline → revisions → final decision → report. The frontend should poll or subscribe to status rather than assuming each operation completes immediately.

39. Example Independent Run Response

POST /evaluations/{id}/agents/run-independent may return: { data: { evaluation_id: "...", status: "PROCESSING", stages: { technical: "RUNNING", hr_culture: "RUNNING", hiring_manager: "RUNNING", skeptic: "RUNNING" } } }. A later GET /agents returns the completed structured evaluations.

40. Example Debate Message

A stored API object should contain: { message_id, round_number: 2, agent_type: "skeptic", message_type: "RESPONSE", statement: "...", responding_to_message_id: "D12", evidence_ids: ["E31"], confidence: "HIGH" }. This makes the direct response relationship machine-verifiable.

41. Example Final Decision Response

The final decision response should expose: recommendation, confidence, confidence_reason, decision_rationale, decision_factors, key_supporting_evidence_ids, key_concern_ids, resolved_disagreements, unresolved_disagreements, and alternative_considered.

42. OpenAPI Requirement

The implementation should generate an OpenAPI 3 specification from typed FastAPI schemas. The OpenAPI document becomes the machine-readable API contract used by frontend development and integration testing.

43. Testing Requirements

For every endpoint, test success and failure paths. Critical tests must verify: independent prompts contain no other agent outputs; debate cannot start early; responses reference real messages; evidence IDs resolve; invalid quotes are rejected; final decision is a separate call; and recommendation generation does not use score averaging.

44. API Completion Criteria

The API is ready when the complete evaluation can be executed through documented endpoints; lifecycle states are enforced; all outputs are structured; evidence is traceable; independent-agent isolation is testable; debate relationships and revisions are queryable; final decisions are separately generated; errors are explicit; and the frontend can build the complete interactive report from API responses alone.